WEEK 11

Saving Data: Room & DataStore

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

So far, everything in your apps disappears when the app is closed. Real apps **remember**: notes, to-do lists, settings, downloaded data for offline use. This week you'll learn **Room** (a database for structured data) and **DataStore** (for small settings), and you'll connect them to ViewModels with Flow. You'll build a **Notes app** that keeps its data.

Day	Topic	You will be able to...
Day 1	Local storage & SQL basics	Choose the right storage and read simple SQL
Day 2	Room: entity, DAO, database	Set up a Room database
Day 3	Repository + ViewModel with Room	Show live database data on screen with Flow
Day 4	More Room	Write search queries, use type converters and handle migrations
Day 5	DataStore for settings	Save and read user preferences
Day 6	Weekly project	Build a Notes app with search, pin and delete

> **Tip:** Room and DataStore couldn't be downloaded here, so their code was compile-checked against stand-ins that copy their APIs. The plain Kotlin examples (like the fake DAO on Day 3) were **run** with real coroutines, and the outputs are real.

WEEK 11 • DAY 1

Local Storage & SQL Basics

Today's goal: Choose between storage options, and understand tables, rows and basic SQL queries.

1. Where can an app save data?

Option	Good for	Example
DataStore	Small key-value settings	Dark mode on/off, username, onboarding completed
Room (SQLite database)	Structured data: many items of the same kind, searching and sorting	Notes, to-dos, contacts, cached products
Files	Large files	Downloaded PDFs, photos
Server	Data shared between devices or users	User account, chat messages

Tip: A common pattern in team apps: load data from the server, **save it in Room**, and show it from Room. The app then works offline and opens instantly. This is called **offline-first**.

2. Tables, rows and columns

A database stores data in **tables**. A table is like a spreadsheet: each **column** is a field, and each **row** is one item.

id (PRIMARY KEY)	title	content	is_pinned	created_at
1	Shopping	Milk, eggs, rice	1	1760600000000
2	Study plan	Finish Week 11	0	1760603600000
3	Ideas	Build a recipe app	0	1760607200000

The **primary key** (id) is unique for every row, so you can always find exactly one item. Room can generate it automatically.

3. SQL in 5 minutes

SQL is the language for asking a database questions. With Room you write small SQL queries inside annotations.

SQL	Meaning
SELECT * FROM notes	Get all columns of all notes
SELECT * FROM notes WHERE id = 2	Get the note with id 2
SELECT * FROM notes ORDER BY created_at DESC	All notes, newest first
SELECT * FROM notes WHERE is_pinned = 1	Only pinned notes
SELECT * FROM notes WHERE title LIKE '%shop%'	Titles containing "shop" (search)
SELECT COUNT(*) FROM notes	How many notes there are
INSERT INTO notes (title, content) VALUES ('A', 'B')	Add a row (Room writes this for you with @Insert)
UPDATE notes SET is_pinned = 1 WHERE id = 3	Change a row (Room: @Update)
DELETE FROM notes WHERE id = 3	Delete a row (Room: @Delete)

You'll mostly write **SELECT** queries yourself. Room generates INSERT, UPDATE and DELETE for you from annotations.

4. Why Room instead of raw SQLite?

- Room **checks your SQL at compile time**: a typo in a column name is an error in Android Studio, not a crash on a user's phone.
- It converts rows into Kotlin **data classes** automatically.
- It supports **suspend functions** (no UI freezes) and **Flow** (the UI updates automatically when data changes).

Practice Exercises

1. Design a table for a to-do app: write the column names and types, and mark the primary key.

2. Write SQL to get all to-dos that are not done, sorted by due date (oldest first).

3. Write SQL to count how many to-dos are done.

4. For each piece of data, choose DataStore, Room or the server: app language, list of saved recipes, user's profile photo shown on all their devices, "has seen the tutorial".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. When should you use DataStore instead of Room?

Q2. What is a primary key?

Q3. Which SQL keyword filters rows?

Q4. What does ORDER BY created_at DESC do?

Q5. What is one big advantage of Room over raw SQLite?

WEEK 11 • DAY 2

Room: Entity, DAO and Database

Today's goal: Add Room to a project and create an entity, a DAO and a database class.

1. The three parts of Room

Sketch: Room's building blocks

```
@Entity   NoteEntity      ->  describes a TABLE (one data class = one table, one object = one row)
@Dao      NoteDao         ->  the QUERIES you can run (functions with SQL)
@Database NotesDatabase   ->  the database itself; gives you the DAOs
```

2. Dependencies

Room uses KSP to generate code (Android only, versions as in your project)

```
// build.gradle.kts (Project): plugins { id("com.google.devtools.ksp") version "2.0.21-1.0.28" apply false }
plugins {
    id("com.google.devtools.ksp")
}
dependencies {
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")          // coroutines and Flow support
    ksp("androidx.room:room-compiler:2.6.1")                 // generates the database code
}
```

Careful: The KSP version must match your Kotlin version (2.0.21 → 2.0.21-1.0.x). A mismatch gives confusing Gradle errors. In a team project, Room is usually already set up.

3. The entity (a table)

NoteEntity.kt

```
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val content: String,
    @ColumnInfo(name = "is_pinned") val isPinned: Boolean = false,
    @ColumnInfo(name = "created_at") val createdAt: Long = System.currentTimeMillis()
)
```

Code	Meaning
@Entity(tableName = "notes")	This data class is a table called notes
@PrimaryKey(autoGenerate = true) val id: Int = 0	Room picks a new unique id when you insert (pass 0 for new notes)
@ColumnInfo(name = "is_pinned")	The column name in the database (snake_case is a common SQL style)
Boolean / Long / String / Int / Double	Supported directly. Other types need a TypeConverter (Day 4)

4. The DAO (the queries)

NoteDao.kt

```
@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY is_pinned DESC, created_at DESC")
    fun observeAll(): Flow<List<NoteEntity>>

    @Query("SELECT * FROM notes WHERE id = :id")
    suspend fun getById(id: Int): NoteEntity?

    @Insert
    suspend fun insert(note: NoteEntity): Long

    @Update
    suspend fun update(note: NoteEntity)

    @Delete
    suspend fun delete(note: NoteEntity)
}
```

Function	Meaning
fun observeAll(): Flow<List<NoteEntity>>	Not suspend: returns a **Flow** that emits the new list **every time the table changes**
suspend fun getById(id: Int): NoteEntity?	One-time read. :id in the SQL is replaced by the parameter. null if not found
@Insert suspend fun insert(...): Long	Adds a row and returns the new id
@Update / @Delete	Finds the row by its primary key, then updates or deletes it

Tip: If you misspell a column in a @Query (for example *is_pined*), the build **fails** with a clear error. That's Room checking your SQL for you.

5. The database class

NotesDatabase.kt

```
@Database(entities = [NoteEntity::class], version = 1, exportSchema = false)
abstract class NotesDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile
        private var instance: NotesDatabase? = null

        fun getInstance(context: Context): NotesDatabase =
            instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    NotesDatabase::class.java,
                    "notes.db"
                ).build().also { instance = it }
            }
    }
}
```

Code	Meaning
@Database(entities = [...], version = 1)	Lists all tables. Increase version when the tables change (Day 4)

Code	Meaning
abstract fun noteDao(): NoteDao	Room generates the implementation
companion object getInstance(...)	A **singleton**: the whole app shares one database instance (creating several is slow and can cause bugs)
@Volatile + synchronized	Makes sure two threads can't create two databases at the same moment
context.applicationContext	Use the app's context, not an Activity's, to avoid memory leaks

With **Hilt** (Week 12), the singleton code disappears: Hilt creates the database once and gives the DAO to whoever needs it.

Practice Exercises

1. Create a **TodoEntity** with id (auto), title, isDone (Boolean, default false) and dueDate (Long).

2. Create a **TodoDao** with: observe all to-dos (not done first), insert, update, delete, and observe only the done ones.

3. Create a **TodoDatabase** class with a getInstance function.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does @Entity describe?

Q2. What is a DAO?

Q3. Why is observeAll() not a suspend function?

Q4. Why should the database be a singleton?

Q5. What does :id mean inside a @Query?

WEEK 11 • DAY 3

Repository + ViewModel with Room

Today's goal: Connect Room to a ViewModel through a repository, and show live data with Flow and stateIn.

1. The full picture

Sketch: Data flows up automatically

```
  NotesScreen  <-- uiState --  NotesViewModel  <-- Flow<List<Note>> --  NotesRepository  <-- Flow --  NoteDao  <-- Room
               -- addNote() -->                -- insert() ------------>                  -- @Insert -->
  Insert a note -> Room updates the table -> the DAO's Flow emits the new list -> the screen recomposes.
```

You never "refresh" the list by hand: Room's Flow emits a new list whenever the table changes, so the screen is always up to date.

2. The repository

NotesRepository

```
data class Note(val id: Int, val title: String, val content: String, val isPinned: Boolean)

fun NoteEntity.toNote() = Note(id, title, content, isPinned)

class NotesRepository(private val dao: NoteDao) {
    val notes: Flow<List<Note>> = dao.observeAll().map { list -> list.map { it.toNote() } }

    suspend fun addNote(title: String, content: String) {
        dao.insert(NoteEntity(title = title.trim(), content = content.trim()))
    }

    suspend fun togglePin(note: Note) {
        val entity = dao.getById(note.id) ?: return
        dao.update(entity.copy(isPinned = !entity.isPinned))
    }

    suspend fun delete(note: Note) {
        dao.getById(note.id)?.let { dao.delete(it) }
    }
}
```

Like the DTO-to-model mapping in Week 10, the repository turns **NoteEntity** (the database shape) into **Note** (the app's model). The ViewModel and UI never see Room classes.

3. See it work without Android: a fake DAO

Because the repository only depends on the NoteDao **interface**, we can give it a fake in-memory DAO. This is exactly how teams test repositories (Week 14), and it lets us run the logic right now:

Repository with a fake DAO (run with real coroutines)

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

data class NoteEntity(val id: Int = 0, val title: String, val isPinned: Boolean = false)

interface NoteDao {
    fun observeAll(): Flow<List<NoteEntity>>
    suspend fun insert(note: NoteEntity): Long
```

```
    suspend fun update(note: NoteEntity)
}

class FakeNoteDao : NoteDao {
    private val table = MutableStateFlow<List<NoteEntity>>(emptyList())
    private var nextId = 1

    override fun observeAll() = table.map { notes -> notes.sortedByDescending { it.isPinned } }
    override suspend fun insert(note: NoteEntity): Long {
        val saved = note.copy(id = nextId++)
        table.value = table.value + saved
        return saved.id.toLong()
    }
    override suspend fun update(note: NoteEntity) {
        table.value = table.value.map { if (it.id == note.id) note else it }
    }
}

fun main() = runBlocking {
    val dao = FakeNoteDao()
    val job = launch {
        dao.observeAll().collect { notes -> println("Screen shows: ${notes.map { it.title }}") }
    }
    delay(100)
    dao.insert(NoteEntity(title = "Shopping"))
    delay(100)
    dao.insert(NoteEntity(title = "Study"))
    delay(100)
    dao.update(NoteEntity(id = 2, title = "Study", isPinned = true))    // pin "Study": it moves to the top
    delay(100)
    job.cancel()
    println("Done")
}
```

Output

```
Screen shows: []
Screen shows: [Shopping]
Screen shows: [Shopping, Study]
Screen shows: [Study, Shopping]
Done
```

Every change to the "table" made the Flow emit a new list, and the "screen" (the collector) printed it. A real Room DAO behaves the same way.

4. The ViewModel

NotesViewModel with stateIn

```
class NotesViewModel(private val repository: NotesRepository) : ViewModel() {

    val notes: StateFlow<List<Note>> = repository.notes.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = emptyList()
    )

    fun addNote(title: String, content: String) {
        if (title.isBlank()) return
        viewModelScope.launch { repository.addNote(title, content) }
    }

    fun togglePin(note: Note) {
        viewModelScope.launch { repository.togglePin(note) }
    }

    fun delete(note: Note) {
        viewModelScope.launch { repository.delete(note) }
    }
}
```

stateIn (Week 9, Day 5) turns the repository's Flow into a StateFlow the screen can collect. Writes (add, pin, delete) run in **viewModelScope.launch** because they're suspend functions. The list updates by itself afterwards.

5. Creating the ViewModel with its repository

viewModel { } with an initializer (until Hilt)

```
@Composable
fun NotesRoute() {
    val context = LocalContext.current
    val viewModel: NotesViewModel = viewModel {
        val dao = NotesDatabase.getInstance(context).noteDao()
        NotesViewModel(NotesRepository(dao))
    }
    val notes by viewModel.notes.collectAsStateWithLifecycle()
    Column {
        notes.forEach { Text(it.title) }
    }
}
```

viewModel { ... } tells Android how to create the ViewModel the first time. After rotation, the same ViewModel is returned and the block doesn't run again. In Week 12, Hilt replaces this with `hiltViewModel()`.

Practice Exercises

1. Write a **TodoRepository** that maps TodoEntity to a Todo model and exposes **val todos: Flow<List<Todo>>**.
2. Extend the fake DAO example with a **delete** function and delete "Shopping" at the end. Run it and check the output.
3. Write a **TodosViewModel** with todos (stateIn), addTodo(title) and toggleDone(todo).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why don't you need to refresh the list after inserting a note?

Q2. Why does the repository map NoteEntity to Note?

Q3. Why can the repository be tested with a fake DAO?

Q4. What does stateIn do here?

Q5. Why are inserts done in viewModelScope.launch?

WEEK 11 • DAY 4

More Room

Today's goal: Write queries with parameters and search, use @Upsert and type converters, and change a database safely with migrations.

1. Queries with parameters

Search, filter and count

```
@Dao
interface NoteQueriesDao {
    @Query("SELECT * FROM notes WHERE title LIKE '%' || :query || '%' OR content LIKE '%' || :query || '%' ORDER BY created_at DESC")
    fun search(query: String): Flow<List<NoteEntity>>

    @Query("SELECT * FROM notes WHERE is_pinned = 1")
    fun observePinned(): Flow<List<NoteEntity>>

    @Query("SELECT COUNT(*) FROM notes")
    fun observeCount(): Flow<Int>

    @Query("DELETE FROM notes WHERE created_at < :before")
    suspend fun deleteOlderThan(before: Long): Int        // returns how many rows were deleted
}
```

'%' || :query || '%' means "anything, then the query, then anything". **||** joins text in SQL. LIKE ignores upper/lower case for English letters.

2. Search that updates as you type

flatMapLatest: switch to a new query's Flow

```
data class Note(val id: Int, val title: String)

interface SearchableNotes {
    fun observeAll(): Flow<List<Note>>
    fun search(query: String): Flow<List<Note>>
}

@OptIn(ExperimentalCoroutinesApi::class)
class NotesSearchViewModel(private val source: SearchableNotes) : ViewModel() {
    private val query = MutableStateFlow("")
    val queryText: StateFlow<String> = query.asStateFlow()

    val notes: StateFlow<List<Note>> = query
        .flatMapLatest { q -> if (q.isBlank()) source.observeAll() else source.search(q) }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())

    fun onQueryChange(text: String) {
        query.value = text
    }
}
```

flatMapLatest switches to a new Flow each time the query changes, and stops listening to the old one. The results are still **live**: if a note is added while you're searching, the results update.

3. @Upsert: insert or update

Insert-or-update in one function

```
@Dao
interface NoteSaveDao {
    @Upsert
    suspend fun upsert(note: NoteEntity)       // id 0 (or new) -> insert; existing id -> update

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(notes: List<NoteEntity>)   // e.g. caching a list from the server
}
```

Tip: @Upsert is perfect for an "edit note" screen that's used both for new notes and for editing existing ones.

4. Type converters: storing other types

Room can store numbers, text and Booleans. For other types (an enum, a date, a list), you tell Room how to convert them with a **TypeConverter**.

Storing an enum

```
enum class Priority { LOW, MEDIUM, HIGH }

class Converters {
    @TypeConverter
    fun priorityToString(priority: Priority): String = priority.name

    @TypeConverter
    fun stringToPriority(value: String): Priority = Priority.valueOf(value)
}

@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val priority: Priority = Priority.MEDIUM
)

@Database(entities = [TaskEntity::class], version = 1)
@TypeConverters(Converters::class)
abstract class TaskDatabase : RoomDatabase()
```

5. Changing the database: migrations

Users already have a database on their phones. If you add a column to an entity, Room must know how to upgrade the old database. Otherwise the app **crashes** on start for every existing user.

A migration from version 1 to 2

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE notes ADD COLUMN color INTEGER NOT NULL DEFAULT 0")
    }
}

fun buildDatabase(context: Context): NotesDatabase =
    Room.databaseBuilder(context.applicationContext, NotesDatabase::class.java, "notes.db")
        .addMigrations(MIGRATION_1_2)
        .build()
```

Step	What to do
1	Change the entity (add val color: Int = 0)
2	Increase version in @Database (1 → 2)
3	Write a Migration with the SQL that changes the old table, and add it with addMigrations
While learning only	`.fallbackToDestructiveMigration()` deletes and recreates the database. **Never in a released app**: users lose their data

6. Look inside the database: Database Inspector

Run your app, then open **View** → **Tool Windows** → **App Inspection** → **Database Inspector**. You can see every table, run SQL queries, and watch rows change live as you use the app. It's the best way to debug Room.

Practice Exercises

1. Add a **search(query)** function to your TodoDao that searches titles.

2. Add a Priority enum to TodoEntity with a TypeConverter, and a query that returns only HIGH priority to-dos.

3. Write a migration that adds a **notes TEXT** column (nullable) to the todos table.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does LIKE '%' || :query || '%' do?

Q2. What does flatMapLatest do in the search ViewModel?

Q3. What does @Upsert do?

Q4. Why are TypeConverters needed?

Q5. What happens if you change an entity without a migration?

WEEK 11 • DAY 5

DataStore for Settings

Today's goal: Save and read small user preferences with Preferences DataStore, and connect them to a settings screen.

1. What is DataStore?

Preferences DataStore stores small **key-value** pairs on the phone: settings like dark mode or the user's name. It's the modern replacement for SharedPreferences. Reading gives you a **Flow**, and writing is a **suspend** function, so it never freezes the UI.

Dependency (Android only)

```
implementation("androidx.datastore:datastore-preferences:1.1.1")
```

2. A settings repository

SettingsRepository.kt

```
val Context.settingsDataStore: DataStore<Preferences> by preferencesDataStore(name = "settings")

data class UserSettings(val darkMode: Boolean = false, val userName: String = "", val fontScale: Int = 100)

class SettingsRepository(private val dataStore: DataStore<Preferences>) {

    private object Keys {
        val DARK_MODE = booleanPreferencesKey("dark_mode")
        val USER_NAME = stringPreferencesKey("user_name")
        val FONT_SCALE = intPreferencesKey("font_scale")
    }

    val settings: Flow<UserSettings> = dataStore.data.map { prefs ->
        UserSettings(
            darkMode = prefs[Keys.DARK_MODE] ?: false,
            userName = prefs[Keys.USER_NAME] ?: "",
            fontScale = prefs[Keys.FONT_SCALE] ?: 100
        )
    }

    suspend fun setDarkMode(enabled: Boolean) {
        dataStore.edit { prefs -> prefs[Keys.DARK_MODE] = enabled }
    }

    suspend fun setUserName(name: String) {
        dataStore.edit { prefs -> prefs[Keys.USER_NAME] = name.trim() }
    }

    suspend fun setFontScale(percent: Int) {
        dataStore.edit { prefs -> prefs[Keys.FONT_SCALE] = percent.coerceIn(80, 150) }
    }
}
```

Code	Meaning
by preferencesDataStore(name = "settings")	Creates one DataStore file called settings, shared by the whole app. Declare it once, at the top level of a file
booleanPreferencesKey("dark_mode")	A typed key: this key always stores a Boolean
dataStore.data.map { prefs -> ... }	A Flow that emits whenever any setting changes. Missing keys give null, so use ?: for defaults

Code	Meaning
dataStore.edit { prefs -> prefs[key] = value }	A suspend function that saves the change safely

3. The ViewModel

SettingsViewModel

```
class SettingsViewModel(private val repository: SettingsRepository) : ViewModel() {

    val settings: StateFlow<UserSettings> = repository.settings.stateIn(
        viewModelScope, SharingStarted.WhileSubscribed(5_000), UserSettings()
    )

    fun onDarkModeChange(enabled: Boolean) {
        viewModelScope.launch { repository.setDarkMode(enabled) }
    }

    fun onUserNameChange(name: String) {
        viewModelScope.launch { repository.setUserName(name) }
    }

    fun onFontScaleChange(percent: Int) {
        viewModelScope.launch { repository.setFontScale(percent) }
    }
}
```

4. The screen

SettingsScreen

```
data class UserSettings(val darkMode: Boolean = false, val userName: String = "", val fontScale: Int = 100)

@Composable
fun SettingsScreen(
    settings: UserSettings,
    onDarkModeChange: (Boolean) -> Unit,
    onUserNameChange: (String) -> Unit,
    onFontScaleChange: (Int) -> Unit
) {
    Column(Modifier.fillMaxSize().padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        Text("Settings", style = MaterialTheme.typography.headlineMedium)
        OutlinedTextField(
            value = settings.userName,
            onValueChange = onUserNameChange,
            label = { Text("Your name") },
            singleLine = true,
            modifier = Modifier.fillMaxWidth()
        )
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Dark mode", modifier = Modifier.weight(1f))
            Switch(checked = settings.darkMode, onCheckedChange = onDarkModeChange)
        }
        Text("Text size: ${settings.fontScale}%")
        Slider(
            value = settings.fontScale.toFloat(),
            onValueChange = { onFontScaleChange(it.roundToInt()) },
            valueRange = 80f..150f
        )
    }
}
```

Close the app completely and open it again: your name, dark mode and text size are still there.

> **Tip:** Saving on every keystroke (as onUserNameChange does) is fine for DataStore with small values. For a long text, keep the text in the ViewModel's state and save it when the user taps Save or leaves the screen.

5. Using a setting app-wide: dark mode

Apply the saved dark mode to the theme (Android only)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val repository = SettingsRepository(applicationContext.settingsDataStore)
        setContent {
            val settings by repository.settings.collectAsStateWithLifecycle(initialValue = UserSettings())
            MyAppTheme(darkTheme = settings.darkMode) {
                AppNavHost()
            }
        }
    }
}
```

6. DataStore or Room?

Question	Use
Is it a single value or a few settings?	DataStore
Is it a list of items you add, edit, search or sort?	Room
Does it need relationships (notes with tags)?	Room
Is it a flag like "onboarding finished"?	DataStore

Practice Exercises

1. Add a **notificationsEnabled** Boolean setting to SettingsRepository, UserSettings and the screen.

2. Add an "onboarding completed" flag, and show a welcome screen only when it's false.

3. Add a "Reset settings" button that clears all preferences (prefs.clear() inside edit).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What kind of data is DataStore for?

Q2. How do you read settings from DataStore?

Q3. How do you write a setting?

Q4. Why use ?: when reading a key?

Q5. Where should preferencesDataStore be declared?

WEEK 11 • DAY 6

Weekly Project: Notes App

Today's goal: Build a Notes app that saves notes in Room, with live search, pinning, editing and deleting.

1. Features

Sketch: The app

```
+--------------------------------------+
| (Q) Search notes                     |
| +----------------------------------+ |
| | Shopping                (*) (x) | |   <- pinned notes first
| | Milk, eggs, rice                 | |
| +----------------------------------+ |
| | Study plan              ( ) (x) | |
| | Finish Week 11 and the Notes app | |
| +----------------------------------+ |
|                            [ + ]     |   <- opens the editor dialog
+--------------------------------------+
```

Feature	How it works
Notes survive closing the app	Saved in Room
The list updates by itself	The DAO returns a Flow
Live search	query + flatMapLatest + a LIKE query
Add and edit with one dialog	editor state in the ViewModel, saved with @Upsert
Pin a note	A SQL UPDATE that flips is_pinned; sorted with ORDER BY is_pinned DESC

2. Data layer: entity, DAO, database, repository

NotesData.kt

```
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val content: String,
    @ColumnInfo(name = "is_pinned") val isPinned: Boolean = false,
    @ColumnInfo(name = "updated_at") val updatedAt: Long = System.currentTimeMillis()
)

@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY is_pinned DESC, updated_at DESC")
    fun observeAll(): Flow<List<NoteEntity>>

    @Query("SELECT * FROM notes WHERE title LIKE '%' || :q || '%' OR content LIKE '%' || :q || '%' ORDER BY is_pinned DESC, updated_at DESC")
    fun search(q: String): Flow<List<NoteEntity>>

    @Upsert
    suspend fun upsert(note: NoteEntity)

    @Query("DELETE FROM notes WHERE id = :id")
    suspend fun deleteById(id: Int)

    @Query("UPDATE notes SET is_pinned = NOT is_pinned WHERE id = :id")
    suspend fun togglePin(id: Int)
}

@Database(entities = [NoteEntity::class], version = 1, exportSchema = false)
abstract class NotesDatabase : RoomDatabase() {
```

```
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile private var instance: NotesDatabase? = null
        fun getInstance(context: Context): NotesDatabase = instance ?: synchronized(this) {
            instance ?: Room.databaseBuilder(context.applicationContext, NotesDatabase::class.java, "notes.db")
                .build().also { instance = it }
        }
    }
}

data class Note(val id: Int, val title: String, val content: String, val isPinned: Boolean)

fun NoteEntity.toNote() = Note(id, title, content, isPinned)

class NotesRepository(private val dao: NoteDao) {
    fun observeNotes(query: String): Flow<List<Note>> =
        (if (query.isBlank()) dao.observeAll() else dao.search(query.trim()))
            .map { list -> list.map { it.toNote() } }

    suspend fun save(id: Int, title: String, content: String) {
        dao.upsert(NoteEntity(id = id, title = title.trim(), content = content.trim()))
    }

    suspend fun delete(id: Int) = dao.deleteById(id)
    suspend fun togglePin(id: Int) = dao.togglePin(id)
}
```

3. ViewModel

NotesViewModel.kt

```
data class NoteEditor(val id: Int = 0, val title: String = "", val content: String = "")

data class NotesUiState(
    val query: String = "",
    val notes: List<Note> = emptyList(),
    val editor: NoteEditor? = null          // null = the editor dialog is closed
)

@OptIn(ExperimentalCoroutinesApi::class)
class NotesViewModel(private val repository: NotesRepository) : ViewModel() {
    private val query = MutableStateFlow("")
    private val editor = MutableStateFlow<NoteEditor?>(null)

    private val notes = query.flatMapLatest { repository.observeNotes(it) }

    val uiState: StateFlow<NotesUiState> = combine(query, notes, editor) { q, list, ed ->
        NotesUiState(query = q, notes = list, editor = ed)
    }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), NotesUiState())

    fun onQueryChange(text: String) { query.value = text }

    fun onAddClick() { editor.value = NoteEditor() }
    fun onNoteClick(note: Note) { editor.value = NoteEditor(note.id, note.title, note.content) }
    fun onEditorTitleChange(text: String) = editor.update { it?.copy(title = text) }
    fun onEditorContentChange(text: String) = editor.update { it?.copy(content = text) }
    fun onEditorDismiss() { editor.value = null }

    fun onEditorSave() {
        val ed = editor.value ?: return
        if (ed.title.isBlank()) return
        viewModelScope.launch {
            repository.save(ed.id, ed.title, ed.content)
            editor.value = null
        }
    }

    fun onDelete(note: Note) { viewModelScope.launch { repository.delete(note.id) } }
    fun onTogglePin(note: Note) { viewModelScope.launch { repository.togglePin(note.id) } }
}
```

Idea	Code
Several flows become one UiState	combine(query, notes, editor) { ... }.stateIn(...)
The editor dialog is state too	editor: NoteEditor? (null = closed)
New and edit share one save	id = 0 means new; @Upsert inserts or updates
Close the dialog only after saving	editor.value = null inside the launch, after repository.save

4. Screen

NotesScreen.kt

```
@Composable
fun NotesRoute() {
    val context = LocalContext.current
    val viewModel: NotesViewModel = viewModel {
        NotesViewModel(NotesRepository(NotesDatabase.getInstance(context).noteDao()))
    }
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    NotesScreen(
```

```
        uiState = uiState,
        onQueryChange = viewModel::onQueryChange,
        onAddClick = viewModel::onAddClick,
        onNoteClick = viewModel::onNoteClick,
        onTogglePin = viewModel::onTogglePin,
        onDelete = viewModel::onDelete,
        onEditorTitleChange = viewModel::onEditorTitleChange,
        onEditorContentChange = viewModel::onEditorContentChange,
        onEditorSave = viewModel::onEditorSave,
        onEditorDismiss = viewModel::onEditorDismiss
    )
}

@Composable
fun NotesScreen(
    uiState: NotesUiState,
    onQueryChange: (String) -> Unit,
    onAddClick: () -> Unit,
    onNoteClick: (Note) -> Unit,
    onTogglePin: (Note) -> Unit,
    onDelete: (Note) -> Unit,
    onEditorTitleChange: (String) -> Unit,
    onEditorContentChange: (String) -> Unit,
    onEditorSave: () -> Unit,
    onEditorDismiss: () -> Unit
) {
    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = onAddClick) { Icon(Icons.Default.Add, contentDescription = "New note") }
        }
    ) { innerPadding ->
        Column(Modifier.padding(innerPadding).fillMaxSize()) {
            OutlinedTextField(
                value = uiState.query,
                onValueChange = onQueryChange,
                placeholder = { Text("Search notes") },
                leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
                singleLine = true,
                modifier = Modifier.fillMaxWidth().padding(16.dp)
            )
            if (uiState.notes.isEmpty()) {
                Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                    Text(if (uiState.query.isBlank()) "No notes yet. Tap + to add one." else "No matching notes")
                }
            } else {
                LazyColumn(contentPadding = PaddingValues(start = 16.dp, end = 16.dp, bottom = 88.dp),
                    verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    items(uiState.notes, key = { it.id }) { note ->
                        NoteCard(note, onClick = { onNoteClick(note) },
                            onTogglePin = { onTogglePin(note) }, onDelete = { onDelete(note) })
                    }
                }
            }
        }
    }

    uiState.editor?.let { editor ->
        AlertDialog(
            onDismissRequest = onEditorDismiss,
            title = { Text(if (editor.id == 0) "New note" else "Edit note") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(editor.title, onEditorTitleChange, label = { Text("Title") }, singleLine = true)
                    OutlinedTextField(editor.content, onEditorContentChange, label = { Text("Content") }, minLines = 3)
                }
            },
            confirmButton = { TextButton(onClick = onEditorSave, enabled = editor.title.isNotBlank()) { Text("Save") } },
            dismissButton = { TextButton(onClick = onEditorDismiss) { Text("Cancel") } }
        )
    }
}

@Composable
fun NoteCard(note: Note, onClick: () -> Unit, onTogglePin: () -> Unit, onDelete: () -> Unit) {
    Card(onClick = onClick, modifier = Modifier.fillMaxWidth()) {
```

```
        Row(Modifier.padding(start = 16.dp, top = 8.dp, bottom = 8.dp), verticalAlignment = Alignment.CenterVertically) {
            Column(Modifier.weight(1f)) {
                Text(note.title, style = MaterialTheme.typography.titleMedium)
                if (note.content.isNotBlank()) {
                    Text(note.content, maxLines = 2, overflow = TextOverflow.Ellipsis,
                        style = MaterialTheme.typography.bodyMedium)
                }
            }
            IconButton(onClick = onTogglePin) {
                Icon(if (note.isPinned) Icons.Filled.Star else Icons.Outlined.Star,
                    contentDescription = if (note.isPinned) "Unpin" else "Pin")
            }
            IconButton(onClick = onDelete) { Icon(Icons.Default.Delete, contentDescription = "Delete") }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun NotesScreenPreview() {
    NotesScreen(
        uiState = NotesUiState(notes = listOf(
            Note(1, "Shopping", "Milk, eggs, rice", isPinned = true),
            Note(2, "Study plan", "Finish Week 11 and the Notes app", isPinned = false)
        )),
        { }, { }, { }, { }, { }, { }, { }, { }, { }
    )
}
```

5. Try it

- Add 3 notes, pin one, delete one, then **close the app completely** (swipe it away) and open it again. Your notes are still there.
- Type in the search box: the list filters as you type.
- Open the **Database Inspector** and watch rows appear and change as you use the app.

6. Project challenges

- Show "Edited 5 min ago" using updatedAt (pass it through to the Note model).
- Add colours: a color column (with a migration!) and a row of colour choices in the editor.
- Add undo for delete: show a Snackbar with Undo, and re-insert the note if tapped.
- Add a DataStore setting for sort order (newest first or A-Z) and use it in the DAO query choice.

Week 11 checklist

I can...	Done?
Choose between DataStore, Room, files and the server	[ ]
Read and write basic SQL queries	[ ]
Create a Room entity, DAO and database	[ ]
Expose Room data as a Flow through a repository	[ ]
Turn repository flows into UiState with stateIn and combine	[ ]
Write search queries, @Upsert and type converters	[ ]
Explain migrations and why destructive migration is dangerous	[ ]
Save and read settings with DataStore	[ ]
Build the Notes app	[ ]

Quick Quiz

Q1. Why does the note list update by itself after saving?

Q2. How does one dialog handle both new and existing notes?

Q3. Why is the editor part of the UiState?

Q4. How are pinned notes shown first?

Q5. Where would you add an Undo for delete?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Local Storage & SQL Basics

Quiz answers

Q1. For small key-value settings, not for lists of structured items.

Q2. A column whose value is unique for each row, used to identify it.

Q3. WHERE

Q4. Sorts the rows by created_at, newest (largest) first.

Q5. SQL queries are checked at compile time (also: data class mapping, suspend and Flow support).

Day 2: Room: Entity, DAO and Database

Exercise 1

```
@Entity(tableName = "todos")
data class TodoEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    @ColumnInfo(name = "is_done") val isDone: Boolean = false,
    @ColumnInfo(name = "due_date") val dueDate: Long
)
```

Exercise 2

```
@Entity(tableName = "todos")
data class TodoEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    @ColumnInfo(name = "is_done") val isDone: Boolean = false,
    @ColumnInfo(name = "due_date") val dueDate: Long
)

@Dao
interface TodoDao {
    @Query("SELECT * FROM todos ORDER BY is_done ASC, due_date ASC")
    fun observeAll(): Flow<List<TodoEntity>>

    @Query("SELECT * FROM todos WHERE is_done = 1")
    fun observeDone(): Flow<List<TodoEntity>>

    @Insert suspend fun insert(todo: TodoEntity)
    @Update suspend fun update(todo: TodoEntity)
    @Delete suspend fun delete(todo: TodoEntity)
}
```

Exercise 3

```
@Entity(tableName = "todos")
data class TodoEntity(@PrimaryKey(autoGenerate = true) val id: Int = 0, val title: String)

@Dao interface TodoDao { @Query("SELECT * FROM todos") fun observeAll(): Flow<List<TodoEntity>> }

@Database(entities = [TodoEntity::class], version = 1, exportSchema = false)
abstract class TodoDatabase : RoomDatabase() {
    abstract fun todoDao(): TodoDao

    companion object {
        @Volatile private var instance: TodoDatabase? = null
        fun getInstance(context: Context): TodoDatabase = instance ?: synchronized(this) {
            instance ?: Room.databaseBuilder(context.applicationContext, TodoDatabase::class.java, "todos.db")
                .build().also { instance = it }
        }
    }
}
```

Quiz answers

Q1. A database table: the data class's properties are its columns.

Q2. A Data Access Object: an interface whose functions are the database queries.

Q3. It returns a Flow, which emits new results every time the table changes.

Q4. Creating several instances is slow and can cause conflicts; the app should share one.

Q5. It's replaced by the function parameter called id.

Day 3: Repository + ViewModel with Room

Exercise 1

```
@Entity(tableName = "todos")
data class TodoEntity(@PrimaryKey(autoGenerate = true) val id: Int = 0, val title: String, val isDone: Boolean = false)

@Dao interface TodoDao {
    @Query("SELECT * FROM todos") fun observeAll(): Flow<List<TodoEntity>>
    @Insert suspend fun insert(todo: TodoEntity)
    @Update suspend fun update(todo: TodoEntity)
}

data class Todo(val id: Int, val title: String, val isDone: Boolean)

class TodoRepository(private val dao: TodoDao) {
    val todos: Flow<List<Todo>> = dao.observeAll().map { list -> list.map { Todo(it.id, it.title, it.isDone) } }
    suspend fun add(title: String) = dao.insert(TodoEntity(title = title))
    suspend fun toggle(todo: Todo) = dao.update(TodoEntity(todo.id, todo.title, !todo.isDone))
}
```

Exercise 2

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

data class NoteEntity(val id: Int = 0, val title: String)

class FakeNoteDao {
    private val table = MutableStateFlow<List<NoteEntity>>(emptyList())
    private var nextId = 1
    fun observeAll(): Flow<List<NoteEntity>> = table
    fun insert(note: NoteEntity) { table.value = table.value + note.copy(id = nextId++) }
    fun delete(id: Int) { table.value = table.value.filter { it.id != id } }
}

fun main() = runBlocking {
    val dao = FakeNoteDao()
    val job = launch { dao.observeAll().collect { println(it.map { n -> n.title }) } }
    delay(50); dao.insert(NoteEntity(title = "Shopping"))
    delay(50); dao.insert(NoteEntity(title = "Study"))
    delay(50); dao.delete(1)
    delay(50); job.cancel()
    println("Done")
}
```

Output

```
[]
[Shopping]
[Shopping, Study]
[Study]
Done
```

Exercise 3

```
data class Todo(val id: Int, val title: String, val isDone: Boolean)

interface TodoSource {
    val todos: Flow<List<Todo>>
    suspend fun add(title: String)
    suspend fun toggle(todo: Todo)
}

class TodosViewModel(private val repository: TodoSource) : ViewModel() {
    val todos: StateFlow<List<Todo>> =
        repository.todos.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())

    fun addTodo(title: String) {
        if (title.isNotBlank()) viewModelScope.launch { repository.add(title) }
    }

    fun toggleDone(todo: Todo) {
        viewModelScope.launch { repository.toggle(todo) }
    }
}
```

Quiz answers

Q1. Room's Flow emits the new list automatically whenever the table changes.

Q2. So the ViewModel and UI don't depend on database classes.

Q3. It depends on the NoteDao interface, so any implementation can be used.

Q4. Turns the repository's Flow into a StateFlow with an initial value for the screen.

Q5. They are suspend functions, so they need a coroutine.

Day 4: More Room

Exercise 1

```
@Entity(tableName = "todos")
data class TodoEntity(@PrimaryKey(autoGenerate = true) val id: Int = 0, val title: String)

@Dao
interface TodoSearchDao {
    @Query("SELECT * FROM todos WHERE title LIKE '%' || :query || '%'")
    fun search(query: String): Flow<List<TodoEntity>>
}
```

Exercise 2

```
enum class Priority { LOW, MEDIUM, HIGH }

class PriorityConverters {
    @TypeConverter fun toText(p: Priority): String = p.name
    @TypeConverter fun fromText(s: String): Priority = Priority.valueOf(s)
}

@Entity(tableName = "todos")
data class TodoEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val priority: Priority = Priority.MEDIUM
)

@Dao
interface TodoPriorityDao {
    @Query("SELECT * FROM todos WHERE priority = 'HIGH'")
    fun observeHighPriority(): Flow<List<TodoEntity>>
}
```

Exercise 3

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE todos ADD COLUMN notes TEXT")
    }
}
```

Quiz answers

Q1. Finds rows where the column contains the query text anywhere.

Q2. Switches to the Flow for the new query and stops listening to the previous one.

Q3. Inserts the row if it's new, or updates it if it already exists.

Q4. Room can only store simple types; converters turn other types (enums, dates) into storable ones.

Q5. Apps with the old database crash on start (unless you use destructive migration, which deletes data).

Day 5: DataStore for Settings

Exercise 1

```
data class UserSettings(val darkMode: Boolean = false, val notificationsEnabled: Boolean = true)

class SettingsRepository(private val dataStore: DataStore<Preferences>) {
    private val NOTIFICATIONS = booleanPreferencesKey("notifications_enabled")
    private val DARK_MODE = booleanPreferencesKey("dark_mode")

    val settings: Flow<UserSettings> = dataStore.data.map {
        UserSettings(darkMode = it[DARK_MODE] ?: false, notificationsEnabled = it[NOTIFICATIONS] ?: true)
    }

    suspend fun setNotifications(enabled: Boolean) {
        dataStore.edit { it[NOTIFICATIONS] = enabled }
    }
}
```

Exercise 2

```
class OnboardingRepository(private val dataStore: DataStore<Preferences>) {
    private val DONE = booleanPreferencesKey("onboarding_done")
    val isOnboardingDone: Flow<Boolean> = dataStore.data.map { it[DONE] ?: false }
    suspend fun markDone() { dataStore.edit { it[DONE] = true } }
}

@Composable
fun StartScreen(onboardingDone: Boolean, onFinishOnboarding: () -> Unit) {
    if (onboardingDone) {
        Text("Home")
    } else {
        Column(Modifier.padding(24.dp)) {
            Text("Welcome!", style = MaterialTheme.typography.headlineMedium)
            Button(onClick = onFinishOnboarding) { Text("Get started") }
        }
    }
}
```

Exercise 3

```
class ResettableSettings(private val dataStore: DataStore<Preferences>) {
    suspend fun reset() {
        dataStore.edit { prefs -> prefs.clear() }
    }
}
```

Quiz answers

Q1. Small key-value settings, such as preferences and flags.

Q2. Through dataStore.data, a Flow of Preferences.

Q3. With the suspend function dataStore.edit { prefs -> prefs[key] = value }.

Q4. A key that was never saved returns null, so ?: gives a default value.

Q5. Once, at the top level of a file, so the whole app shares one DataStore.

Day 6: Weekly Project: Notes App

Quiz answers

Q1. The DAO returns a Flow, which Room re-emits when the table changes.

Q2. NoteEditor.id is 0 for new notes, and @Upsert inserts or updates.

Q3. So the dialog (and what was typed) survives rotation and is controlled by the ViewModel.

Q4. ORDER BY is_pinned DESC in the SQL query.

Q5. Keep the deleted note in the ViewModel, show a Snackbar, and re-insert it with upsert if Undo is tapped.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 12** material. Great work, Anjana!